

VIBE - Project Summary

PROJECT OVERVIEW

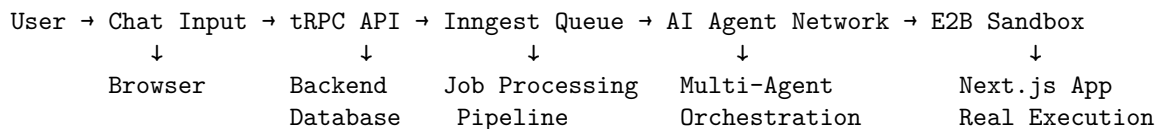
Vibe is an AI-powered development platform that enables users to create web applications by conversing with intelligent AI agents. The system dynamically generates Next.js applications in real-time sandboxes, providing a live preview and code editing interface. This is essentially an “AI-powered app builder” where users describe what they want, and AI agents build it for them in isolated environments.

Core Value Proposition

- **Chat-based Development:** Describe your app idea via natural language
- **Real-time Code Generation:** AI agents write production-quality Next.js code
- **Live Sandboxes:** Run and preview generated applications instantly
- **Persistent Projects:** Save multiple projects, build upon them iteratively
- **Rate-limited Access:** Manage usage with credit-based system

ARCHITECTURE OVERVIEW

High-Level Flow



System Components

1. **Frontend (Next.js 15):** React 19 UI with real-time updates
2. **Backend API:** tRPC for type-safe RPC communication
3. **Job Queue:** Inngest for async processing and agent orchestration
4. **AI Agents:** OpenAI-powered multi-agent system for code generation
5. **Sandbox:** E2B Code Interpreter for isolated app execution
6. **Database:** PostgreSQL with Prisma ORM for data persistence
7. **Authentication:** Clerk for user management
8. **UI Components:** Shadcn/ui + Radix UI for consistent design

TECHNOLOGY STACK

Frontend

- **Next.js 15.3.3** - React framework with App Router

- **React 19** - Latest React with automatic batching
- **TypeScript** - Type safety across the project
- **Tailwind CSS v4** - Utility-first CSS framework
- **Shadcn/ui** - Pre-built component library (Radix UI based)
- **React Query (@tanstack/react-query)** - Client-side data caching
- **React Hook Form** - Lightweight form management
- **Lucide React** - Icon library

Backend & Services

- **tRPC** - Type-safe API layer (client & server)
- **Prisma 6.10.1** - ORM for database operations
- **PostgreSQL** - Primary data store
- **Clerk** - Authentication & user management

AI & Execution

- **@inngest/agent-kit** - Multi-agent orchestration framework
- **OpenAI API** (GPT-4.1, GPT-4o) - LLM for agents
- **@e2b/code-interpreter** - Isolated sandbox environment
- **Inngest 3.39.2** - Event-driven job queue

Additional Libraries

- **Zod** - Schema validation
- **SuperJSON** - JSON serialization for complex types
- **Sonner** - Toast notifications
- **Next-themes** - Dark mode support

DATABASE SCHEMA

Models

1. Project

```
{
  id: String (UUID) - Primary key
  name: String - Auto-generated slug name
  userId: String - Clerk user ID
  createdAt: DateTime
  updatedAt: DateTime
  messages: Message[] - Related messages
}
```

Purpose: Represents a user's project/app

2. Message

```
{
  id: String (UUID) - Primary key
  content: String - Chat message content
  role: "USER" | "ASSISTANT" - Message sender
  type: "RESULT" | "ERROR" - Message type
  createdAt: DateTime
  updatedAt: DateTime
  projectId: String (FK) - Related project
  fragment: Fragment? - Generated app files
}
```

Purpose: Stores conversation history between user and AI

3. Fragment

```
{
  id: String (UUID) - Primary key
  messageId: String (UQ, FK) - One-to-one with Message
  sandboxUrl: String - Live preview URL
  title: String - Generated fragment title
  files: Json - File structure {path: content}
  createdAt: DateTime
  updatedAt: DateTime
}
```

Purpose: Stores generated code artifacts and sandbox URL

4. Usage

```
{
  key: String (PK) - User ID or session key
  points: Int - Credit balance
  expire: DateTime? - Optional expiration
}
```

Purpose: Rate limiting & credit tracking

Relationships

User (Clerk)

- Project (one-to-many)
- Message (one-to-many)
- Fragment (one-to-one)
- Usage (one-to-one via key)

API ENDPOINTS (tRPC)

Projects Router

projects.getOne (Query - Protected)

- **Input:** { id: string }
- **Output:** Project object
- **Purpose:** Fetch specific project details

projects.getMany (Query - Protected)

- **Input:** None
- **Output:** Project[]
- **Purpose:** List all user's projects

projects.create (Mutation - Protected)

- **Input:** { value: string } (user request)
- **Output:** Project
- **Flow:**
 1. Check credit balance
 2. Create project with auto-generated name
 3. Create initial USER message
 4. Send `code-agent/run` event to Inngest
 5. Return project details

Messages Router

messages.getMany (Query - Protected)

- **Input:** { projectId: string }
- **Output:** Message[] (includes fragment)
- **Purpose:** Fetch conversation history

messages.create (Mutation - Protected)

- **Input:** { value: string, projectId: string }
- **Output:** Message
- **Flow:**
 1. Verify project ownership
 2. Check credit balance
 3. Create USER message
 4. Send `code-agent/run` event to Inngest
 5. Background agent processes request

Usage Router

usage.status (Query - Protected)

- **Input:** None
- **Output:** Usage status object

- **Purpose:** Get current credit balance
-

INNGEST JOB QUEUE & AGENTS

Job Event: code-agent/run

Triggered by:

- User creates a project
- User sends a message in a project

Event Payload:

```
{
  value: string,           // User's prompt/request
  projectId: string,       // Which project to work on
}
```

Agent Execution Pipeline

Step 1: Sandbox Setup

```
get-sandbox-id
  Create E2B Sandbox (vibe-nextjs-test-2 template)
  Set 30-minute timeout
  Return sandboxId
```

Step 2: Context Gathering

```
get-previous-messages
  Query last 5 messages from database
  Format as Message[] with role (user/assistant)
  Initialize agent state with message history
```

Step 3: Agent State Creation

```
AgentState {
  summary: string,          // Agent's final task summary
  files: {
    [path: string]: string // Generated files with content
  }
}
```

Step 4: Code Agent Setup Creates a **Code Agent** (GPT-4.1) with 3 tools:

Tool 1: Terminal

- Execute shell commands in sandbox

- Returns stdout/stderr
- Used for: npm install, running scripts

Tool 2: CreateOrUpdateFiles

- Write files to sandbox
- Updates state.files dictionary
- Used for: Creating React components, pages, utilities

Tool 3: ReadFiles

- Read file contents from sandbox
- Returns {path, content} pairs
- Used for: File verification, understanding structure

Step 5: Agentic Loop (max 15 iterations)

Loop until <task_summary> found:

1. Send to LLM (GPT-4.1):
 - Previous messages (context)
 - Available tools
 - Current state
2. LLM decides: Which tool to use? Or generate summary?
3. If tool chosen:
 - Execute tool (terminal/file ops)
 - Update state with results
 - Continue loop
4. If summary found:
 - Exit loop
 - Proceed to post-processing

Step 6: Fragment Generation Fragment Title Generator (GPT-4o)

- Receives: `state.data.summary`
- Prompt: Generate 3-word max title
- Output: Parsed title (e.g., "Landing Page")

Response Generator (GPT-4o)

- Receives: `state.data.summary`
- Prompt: Generate 1-3 sentence user-friendly explanation
- Output: Parsed response text

Step 7: Sandbox Deployment

`get-sandbox-url`

Connect to active sandbox

Get host URL for port 3000

Return `https://sandbox-host-id`

Step 8: Result Persistence

```
save-result (if no error):  
  1. Create ASSISTANT Message  
  2. Create Fragment with:  
    - sandboxUrl  
    - title (from generator)  
    - files (from agent state)  
  3. Save to database
```

Step 9: Return Result

```
{  
  url: string,           // Live preview URL  
  title: string,         // Generated title  
  files: Record<string, string>, // File structure  
  summary: string        // Agent's summary  
}
```

Error Handling

- If no summary or no files: Create ERROR message
 - Error message saved to database
 - User notified in UI
-

AUTHENTICATION FLOW

Clerk Integration

1. **User Signs Up/In:** Redirected to Clerk authentication
2. **Session Creation:** Clerk middleware validates session
3. **User Context:** `auth()` provides `userId`
4. **Protected Routes:**
 - Public: `/`, `/sign-in`, `/sign-up`, `/api`, `/pricing`
 - Protected: `/projects/*`

Middleware Protection

```
const isPublicRoute = createRouteMatcher([  
  "/",  
  "/sign-in(.*)",  
  "/sign-up(.*)",  
  "/api(.*)",  
  "/pricing(.*)",  
]);
```

// Routes not in this list require authentication

RATE LIMITING & CREDITS

Credit System

- **Storage:** Usage table with key + points
- **Consumed:** On project creation, on each message
- **Checked:** Before Inngest event fired

Flow

```
try {
  await consumeCredits() // Deduct points
  // Proceed with project/message creation
} catch (error) {
  if (error instanceof Error)
    → BAD_REQUEST (system error)
  else
    → TOO_MANY_REQUESTS (out of credits)
}
```

USER INTERFACE STRUCTURE

Page Hierarchy

Home Page (/)

- Logo
- Hero Section ("Build something with Vibe")
- Project Form (chat input)
- Projects List (paginated)

Project Page (/projects/[projectId])

ProjectView

ProjectHeader (project name, etc)

MessagesContainer

Message cards (alternating user/assistant)

Message types:

- User message
- Loading spinner
- Assistant message + Fragment preview

MessageForm (new chat input)

Fragment Web Preview (live iframe)
File Explorer (code tree)

Components

ProjectForm

- Text input for initial project request
- Submits via `projects.create` mutation
- Creates project + triggers Inngest

ProjectsList

- Lists user's projects
- Click to open project detail
- Auto-fetches via tRPC

MessageCard

- Displays individual message
- Shows Fragment preview if available
- Links to live sandbox URL

FragmentWeb

- Iframe rendering sandbox URL
- Live preview of generated app

FileExplorer

- Displays generated file structure
- Syntax highlighting with Prismjs
- Theme support (light/dark)

DATA FLOW DIAGRAM

User Creates Project

1. User enters prompt in ProjectForm
2. Form submits via `trpc.projects.create()`
3. Backend creates Project + initial Message
4. `inngest.send("code-agent/run", {...})`
5. Inngest queue receives event (asynchronous)
6. Backend returns Project to frontend
7. Frontend redirects to `/projects/[projectId]`
8. Frontend renders ProjectView with prefetched data
9. Inngest triggers code-agent (background)
10. Agent executes (sandbox, tools, LLM, etc.)
11. Agent saves Fragment + Message to DB

12. Frontend polls/subscribes for updates
13. User sees generated app + code

User Sends Message

1. User types follow-up in MessageForm
 2. Form submits via `trpc.messages.create()`
 3. Backend creates Message (USER role)
 4. `inngest.send("code-agent/run", {...})`
 5. Backend returns Message to frontend
 6. Frontend adds message to list
 7. Frontend shows loading spinner
 8. Inngest triggers code-agent
 9. Agent processes (context: last 5 messages)
 10. Agent generates/modifies Next.js app
 11. Fragment saved to existing Message
 12. Frontend fetches updated messages
 13. User sees new/updated app + code
-

PROJECT FILE STRUCTURE

```
gruadet project 2/
src/
  app/
    (home)/
      page.tsx
      pricing/
      sign-in/
      sign-up/
    projects/[projectId]/
      page.tsx
    api/
      inngest/
      trpc/[trpc]/
    layout.tsx
    error.tsx
    globals.css

  components/
    ui/
    code-view/
    file-explorer.tsx
    hint.tsx
    tree-view.tsx

# Next.js app routes
# Home page group
# Home page
# Pricing page
# Clerk auth
# Project detail
# Inngest webhook
# tRPC endpoint
# Root layout
# Error boundary

# Shared UI components
# Shadcn components
# Code display
```

hooks/	# Custom React hooks
use-current-theme.ts	
use-mobile.ts	
use-scroll.ts	
inngest/	# Job queue setup
client.ts	# Inngest client
functions.ts	# Code agent function
types.ts	# Constants
utils.ts	# Helper functions
lib/	# Utilities
db.ts	# Prisma client
usage.ts	# Credit management
utils.ts	# General utils (cn, etc)
modules/	# Feature modules
home/	# Home page logic
constants.ts	
ui/components/	
projects/	# Project page logic
server/procedures.ts	# tRPC router
ui/	# Project UI
messages/	# Message logic
server/procedures.ts	# tRPC router
server/	
usage/	# Usage tracking
server/procedures.ts	
trpc/	# tRPC setup
init.ts	# tRPC initialization
server.tsx	# Server provider
client.tsx	# Client provider
query-client.ts	
routers/_app.ts	# Main router
middleware.ts	# Clerk auth middleware
prompt.ts	# Agent system prompts
types.ts	# Global types
prisma/	# Database schema
schema.prisma	
sandbox-templates/	# E2B template
nextjs/	
e2b.Dockerfile	

```
compile_page.sh

public/                                # Static assets
  logo.svg
```

HOW EVERYTHING WORKS TOGETHER

Scenario: User Creates Project with Request “Build a todo app”

Phase 1: Request Reception

1. User enters “Build a todo app” in ProjectForm
2. Form validation (min 1 char, max 10000)
3. `trpc.projects.create()` called with value

Phase 2: Backend Processing

1. **Authentication:** Clerk middleware verifies user
2. **Authorization:** Check if user is authenticated
3. **Credit Check:** `consumeCredits()` deducts points
4. **Project Creation:**
 - Generate slug name (e.g., “rapid-buffalo”)
 - Create Project in DB
 - Create initial USER Message
5. **Event Dispatch:** `inngest.send("code-agent/run", {...})`
6. **Response:** Return Project + Message to frontend

Phase 3: Frontend Update

1. Form submission completes
2. `redirect("/projects/[projectId]")` triggered
3. ProjectView page loads
4. Pre-fetches messages + project via tRPC
5. Initial UI renders with USER message visible

Phase 4: Inngest Processing (Asynchronous)

1. Inngest webhook receives event
2. `codeAgentFunction` triggered
3. **Sandbox Creation:**
 - Spawn E2B sandbox
 - Set 30-min timeout
4. **Context Loading:**
 - Fetch last 5 messages (empty initially)
 - Initialize agent state

Phase 5: Code Agent Loop

1. **Iteration 1-N** (max 15):
 - Send prompt + messages to GPT-4.1
 - LLM decides: What tool to use?

Example Iteration A:

 - Decide: Need to create pages
 - Call `createOrUpdateFiles` tool
 - Write `app/page.tsx` with todo form
 - Update `state.files`

Example Iteration B:

 - Need to install UI library
 - Call `terminal` tool
 - Run `npm install some-ui-library`
 - Check success in `stdout`

Example Iteration C:

 - Verify component structure
 - Call `readFiles` tool
 - Read `app/page.tsx`
 - Check implementation
2. **Summary Detection:**
 - Agent generates `<task_summary>` tag
 - Loop exits after ~5-10 iterations
 - State contains complete Next.js app

Phase 6: Post-Generation

1. **Fragment Title** (GPT-4o):
 - Input: Task summary
 - Output: “Todo App”
2. **Response** (GPT-4o):
 - Input: Task summary
 - Output: “Built a fully functional todo app with add, edit, and delete features!”
3. **Sandbox URL:**
 - Get host for port 3000
 - Returns `https://sandbox-abcd1234.e2b.dev`

Phase 7: Database Persistence

1. Create ASSISTANT Message
2. Create Fragment with:
 - `sandboxUrl`: Live preview link
 - `title`: “Todo App”
 - `files`: Complete file structure
3. Save to DB

Phase 8: Frontend Updates (Real-time)

1. Frontend polls or subscribes for message updates
2. Shows Message from ASSISTANT
3. Displays Fragment preview (iframe)
4. Shows file explorer with generated code
5. User can click “View App” to see live preview

Phase 9: User Refinement

1. User sends follow-up: “Add categories to todos”
 2. Steps 2-8 repeat with context
 3. Agent sees previous messages (including todo app code)
 4. Agent modifies existing files
 5. New Fragment with updated code
-

KEY TECHNOLOGIES IN ACTION

Inngest (Job Orchestration)

- **Why:** Handle async AI processing without blocking users
- **How:** Event-driven, durable job queue
- **Benefit:** Decouples frontend request from backend work

E2B Sandbox (Isolated Execution)

- **Why:** Run untrusted AI-generated code safely
- **How:** Docker-based sandboxes with file system + terminal access
- **Benefit:** App preview + development environment in one

tRPC (Type-Safe API)

- **Why:** Share TypeScript types between frontend/backend
- **How:** Type inference from backend procedures
- **Benefit:** Compile-time safety, auto-complete, no schema drift

Prisma ORM (Database)

- **Why:** Type-safe database access
- **How:** Schema-driven migrations + generated client
- **Benefit:** Avoid SQL injection, type safety, easy relationships

Clerk (Auth)

- **Why:** Manage user authentication
- **How:** Pre-built auth UI + middleware
- **Benefit:** Secure, GDPR-compliant, minimal setup

OpenAI Agents (Code Generation)

- **Why:** Generate production-quality Next.js code
- **How:** Multi-agent system, tool use (terminal, file ops)
- **Benefit:** Full-stack implementation, iterative refinement

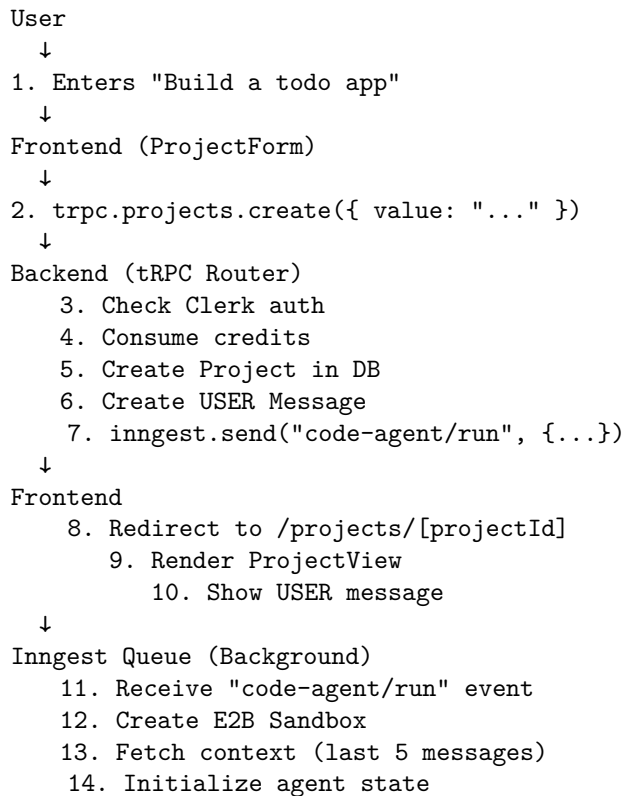
Next.js (Web Framework)

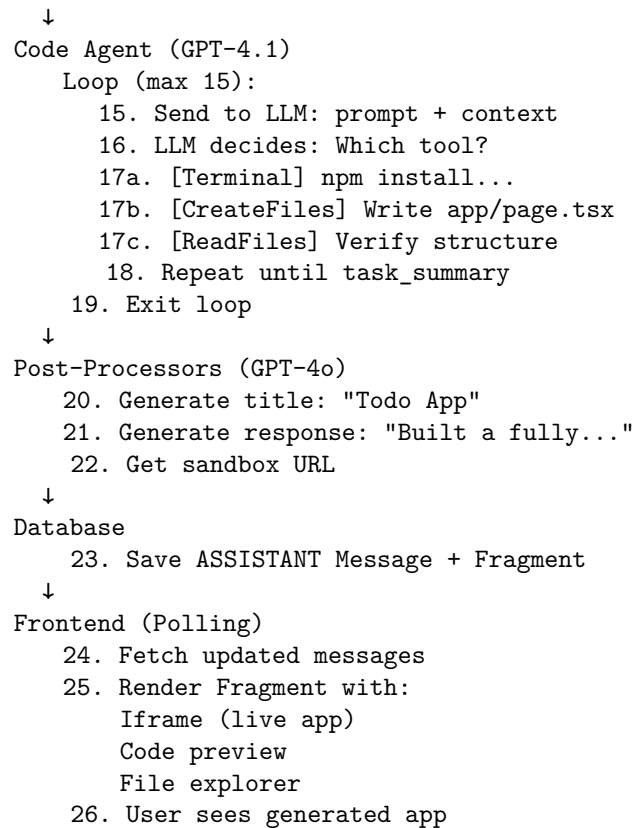
- **Why:** React framework for both frontend + backend
- **How:** App Router for type-safe pages, API routes
- **Benefit:** Unified development, full-stack capabilities

Shadcn/UI (Components)

- **Why:** Accessible, composable component library
 - **How:** Copy-paste components, fully customizable
 - **Benefit:** Professional UI without reinventing the wheel
-

SEQUENCE: Full Request Lifecycle





KEY INSIGHTS

Why This Architecture?

1. **Asynchronous Processing:** AI work doesn't block user interactions
2. **Sandboxed Execution:** Generated code runs safely
3. **Type Safety:** TypeScript + tRPC prevent bugs
4. **Scalability:** Inngest can handle thousands of jobs
5. **Persistence:** Database stores history for refinement
6. **Multi-Agent:** Different models for different tasks (GPT-4.1 for coding, GPT-4o for generation)

User Experience Benefits

1. **Real-time Feedback:** Sandbox preview updates live
2. **Iterative Building:** Refinement through conversation
3. **Full Control:** Access generated code directly
4. **Rate Limiting:** Prevents abuse via credit system

5. **Persistent Projects:** Complete project history saved

Developer Experience

1. **Type Safety:** TypeScript everywhere
 2. **Full-Stack:** React + Node in one codebase
 3. **Composable Components:** Shadcn/ui for rapid UI
 4. **Easy Deployment:** Next.js handles production builds
 5. **Observable:** Inngest dashboards for job monitoring
-

COMMON WORKFLOWS

Workflow 1: Creating a Project

User → ProjectForm → tRPC → DB → Inngest → Agent → Sandbox → Fragment → UI
Time: ~5-30 seconds depending on complexity

Workflow 2: Refining a Project

User → MessageForm → tRPC → DB → Inngest → Agent (+ context) → Sandbox → Fragment → UI
Time: ~5-30 seconds, faster due to existing context

Workflow 3: Viewing Generated App

Fragment saved → Frontend iframe → E2B URL → Live app renders in browser
Time: Instant after generation completes

Workflow 4: Code Inspection

Fragment files → File Explorer → CodeView component → Prismjs highlighting
Time: Instant

Important Constraints & Limitations

1. **Sandbox Timeout:** 30 minutes max per session
 2. **Agent Loop:** Capped at 15 iterations
 3. **Message History:** Only last 5 messages for context
 4. **File Paths:** Must use relative paths (no absolute paths)
 5. **Styling:** Tailwind CSS only (no plain CSS)
 6. **Package Installs:** Must use terminal tool
 7. **File Size:** Max 10MB per spec file
-

POTENTIAL EXTENSIONS

1. **Streaming Responses:** Real-time agent output to frontend
 2. **Version Control:** Commit/rollback functionality
 3. **Collaboration:** Multi-user editing
 4. **Export:** Download generated apps
 5. **Analytics:** Track usage patterns
 6. **Templates:** Pre-built templates for common apps
 7. **Custom Models:** Support for other LLMs
 8. **Custom Sandboxes:** User-defined runtimes
-

DEPENDENCIES SUMMARY

Category	Tools	Purpose
Frontend	React 19, Next.js 15, TypeScript	UI framework
Styling	Tailwind CSS v4, Shadcn/ui, Radix UI	Design system
State	React Query, tRPC	Data management
Forms	React Hook Form, Zod	Form handling
Backend	Node.js, tRPC, Express (via Next.js)	API server
Database	PostgreSQL, Prisma	Data persistence
Auth	Clerk	User management
Jobs	Inngest	Async processing
AI	OpenAI GPT-4.1, GPT-4o	Code generation
Sandbox	E2B Code Interpreter	App execution

Key Takeaways

Vibe is a sophisticated full-stack AI application that:

1. **Accepts natural language** requests from users
2. **Delegates to AI agents** for code generation
3. **Executes in isolated sandboxes** for safety
4. **Displays live previews** immediately
5. **Persists everything** in a database
6. **Supports iterative refinement** through conversation
7. **Manages resources** via credit system
8. **Uses modern tech stack** for scalability
9. **Prioritizes type safety** throughout
10. **Provides excellent UX** with real-time feedback

The architecture is production-ready and demonstrates best practices in full-stack development, AI integration, and cloud-based sandboxing.